Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

Machine Learning & Deep Learning

03: CLUSTERING

13. April 2026

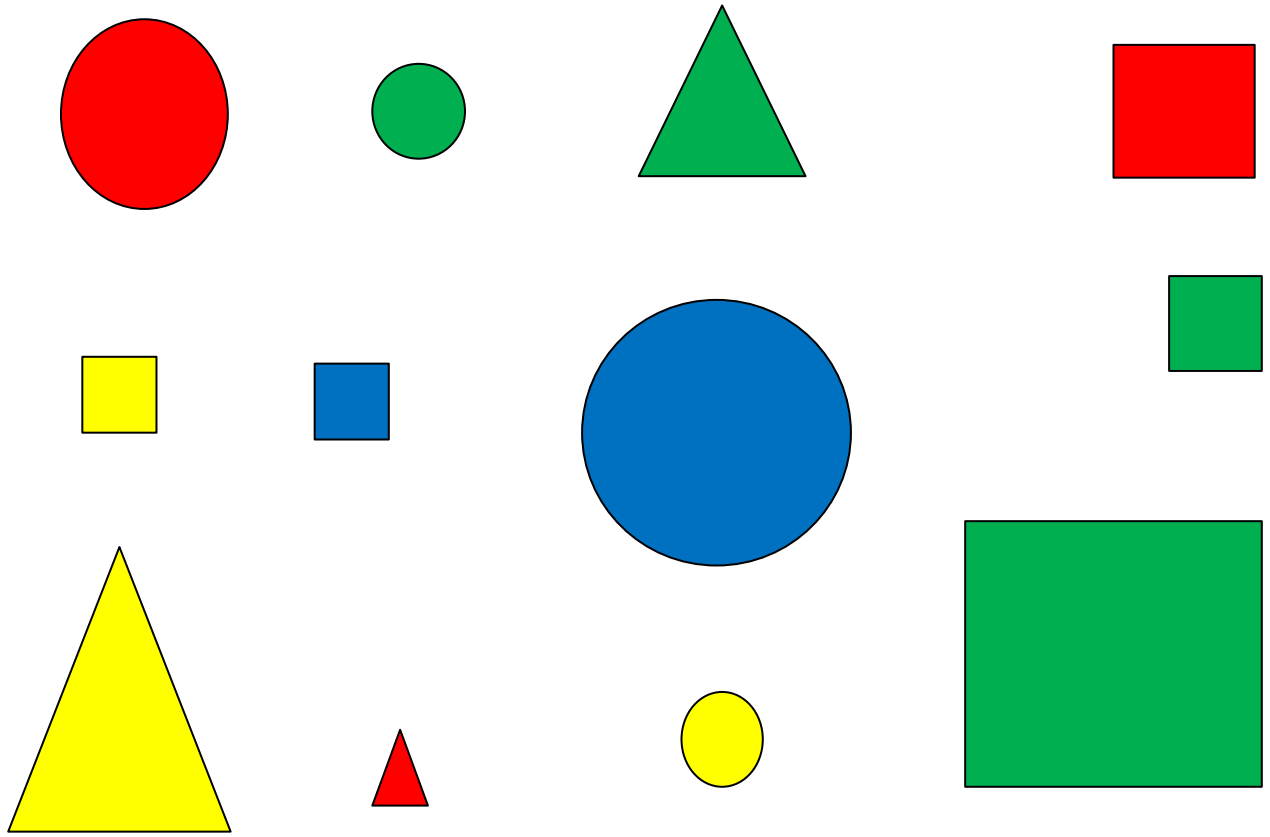
Motivation

- Supervised machine learning
  - Set of labeled examples to learn from: training data
  - Develop model from training data
  - Use model to make predictions about new data

- **Unsupervised machine learning**
  - **Unlabeled data, look for patterns or structure(similar to data mining)**

Motivation

How would you group/cluster these objects?



Motivation

Unlike classification, there is no label

- **Medical patients**
  - Feature values: age, gender, symptom1-severity, symptom2-severity, test-result1, test-result2
- **Web pages**
  - Feature values: URL domain, length, #images, heading1,heading2, ..., heading n
- **Products**
  - Feature values: category, name, size, weight, price

Motivation

- Classification
  - Assign labels to clusters
  - Now have labeled training data for future classification
- Identify similar items
  - For substitutes or recommendations
  - For de-duplication
- Anomaly (outlier) detection
  - Items that are far from any cluster

Motivation

Like K-nearest neighbors, for any pair of data items i1 and i2, from their feature values can compute distance function:

$$distance(\mathrm{i}_1, \mathrm{i}_2)$$

Example:

Features - gender, profession, age, income, postal-code

person1 = (male, teacher, 47, $25K, 94305)

person2 = (female, teacher, 43, $28K, 94309)

$$distance(person1, person2)$$

distance() can be defined as inverse of similarity()

Motivation

Cluster Analysis (or simply “clustering”):

Partitioning a set of data objects into groups (=clusters), such that:

- data objects within one cluster are similar to each other
- data objects of different clusters are dissimilar to each other

Goal:
Discover previously unknown groups within the data.

- Sometimes number of clusters is pre-specified
- Typically clusters need not be same size

Clustering is known as unsupervised learning because no information about classes is available for the instances

Motivation

The most common types of clustering methods are

1. partitioning methods
2. hierarchical methods
3. density-based methods
4. (grid-based methods)

we will focus on partitioning, hierarchical and density methods

Partitioning methods

- find k clusters in the data set (k has to be pre-defined !)
- each cluster must contain ≥1 instances
- each instance must belong to exactly one cluster
- usually distance-based

Steps:

1. Creation of initial partitioning (e.g. randomly).
2. Iterative improvement of the partitioning by moving objects from one cluster to another. This is done by optimizing some criterion of what a "good" partitioning should look like.
3. Stop, if the partitioning quality criterion is satisfied.

Partitioning methods

K-means

The K-means algorithm for partitioning, where each cluster`s center is represented by the mean value of the objects in the cluster

Input:

- K: the number if clusters
- D: a data set containing n objects

Output:

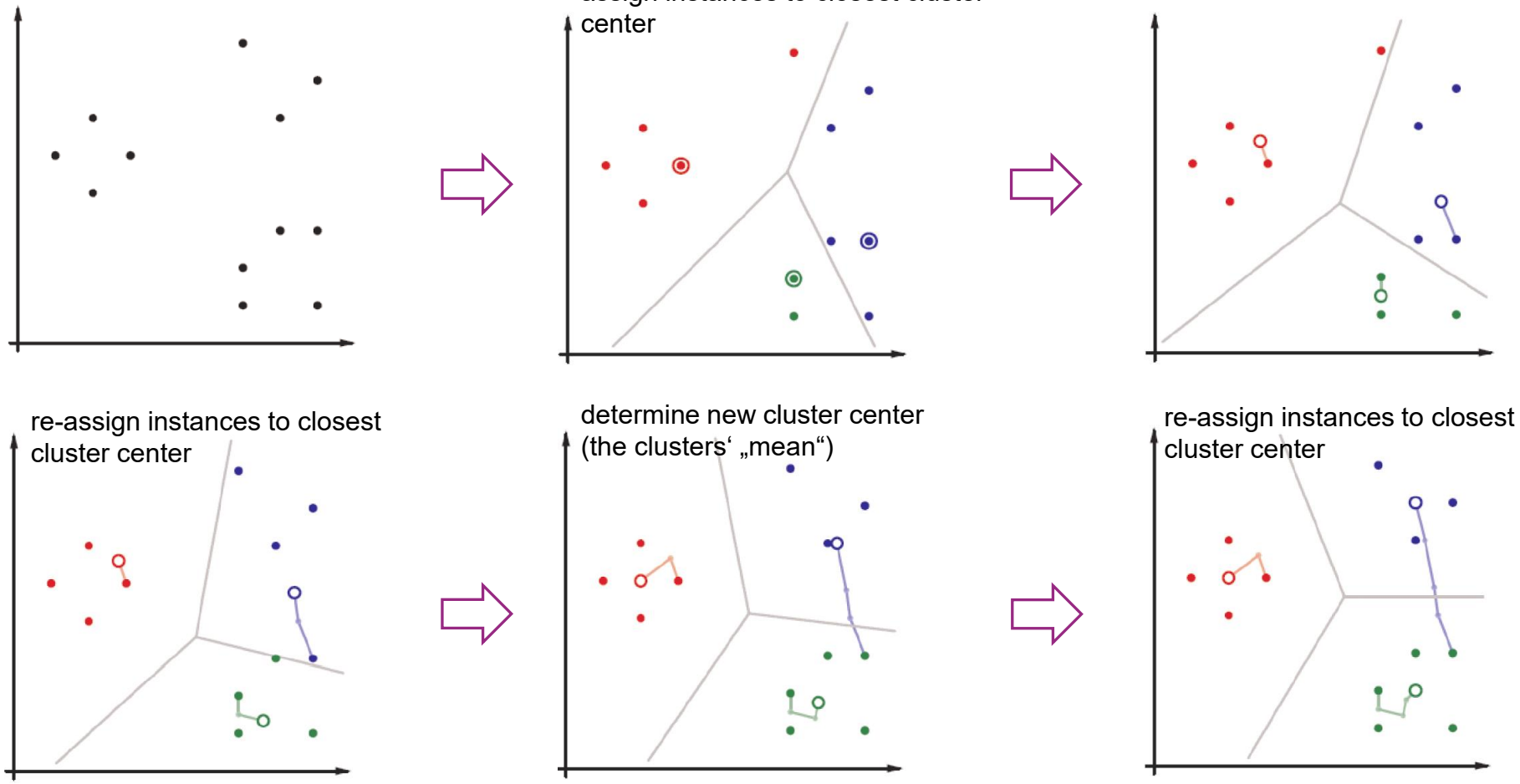
- A set of k clusters

Method:

1. Randomly choose k objects from D as the initial cluster centers.
2. Assign each object to the cluster whose mean is closest to the object.
3. Update the cluster means by computing the mean of the objects in each cluster.
4. Repeat steps 2 and 3 until no changes occur.

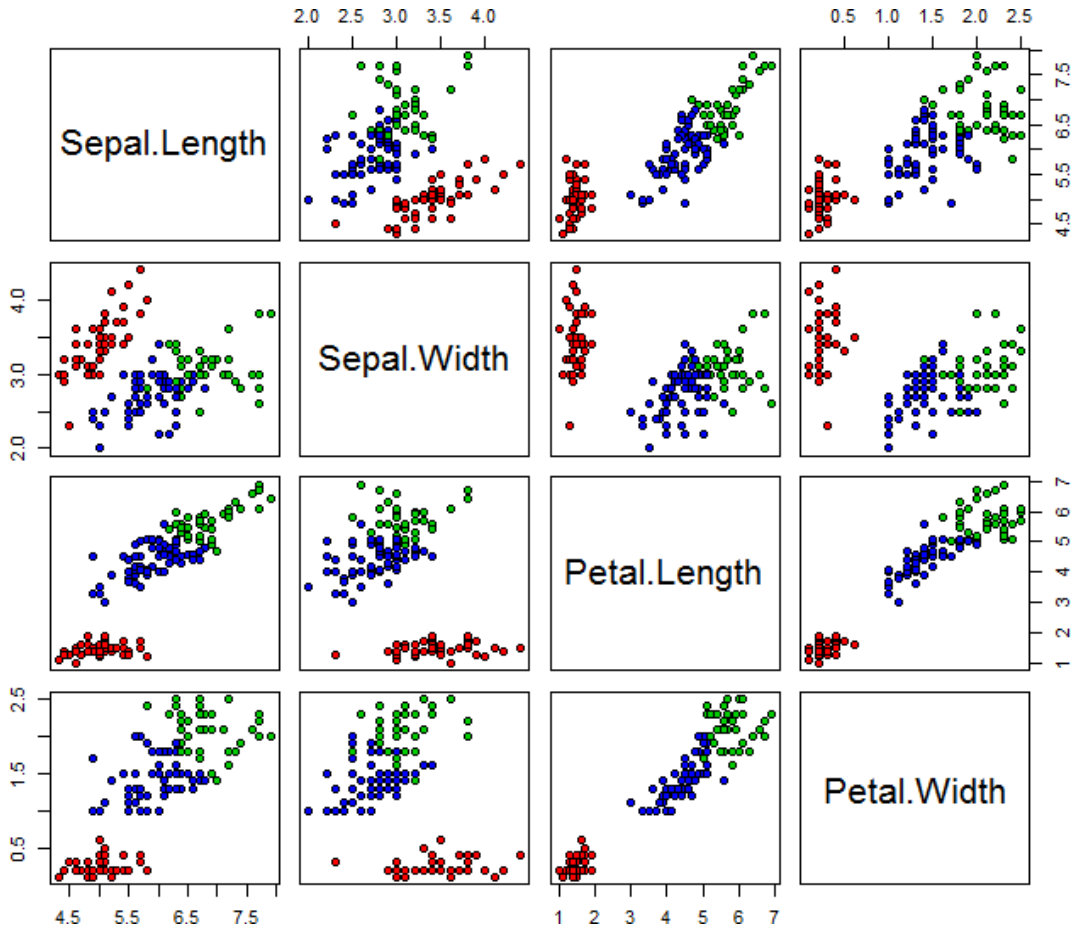
Partitioning methods

K-means



Partitioning methods
K-means

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import Kmeans
from sklearn.preprocessing import MinMaxScaler

# Load the Iris data set
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Normalize the data
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(df)


# Train k-means
kmeans = KMeans(n_clusters=3, random_state=42, max_iter=200)
kmeans.fit(X_scaled)
```

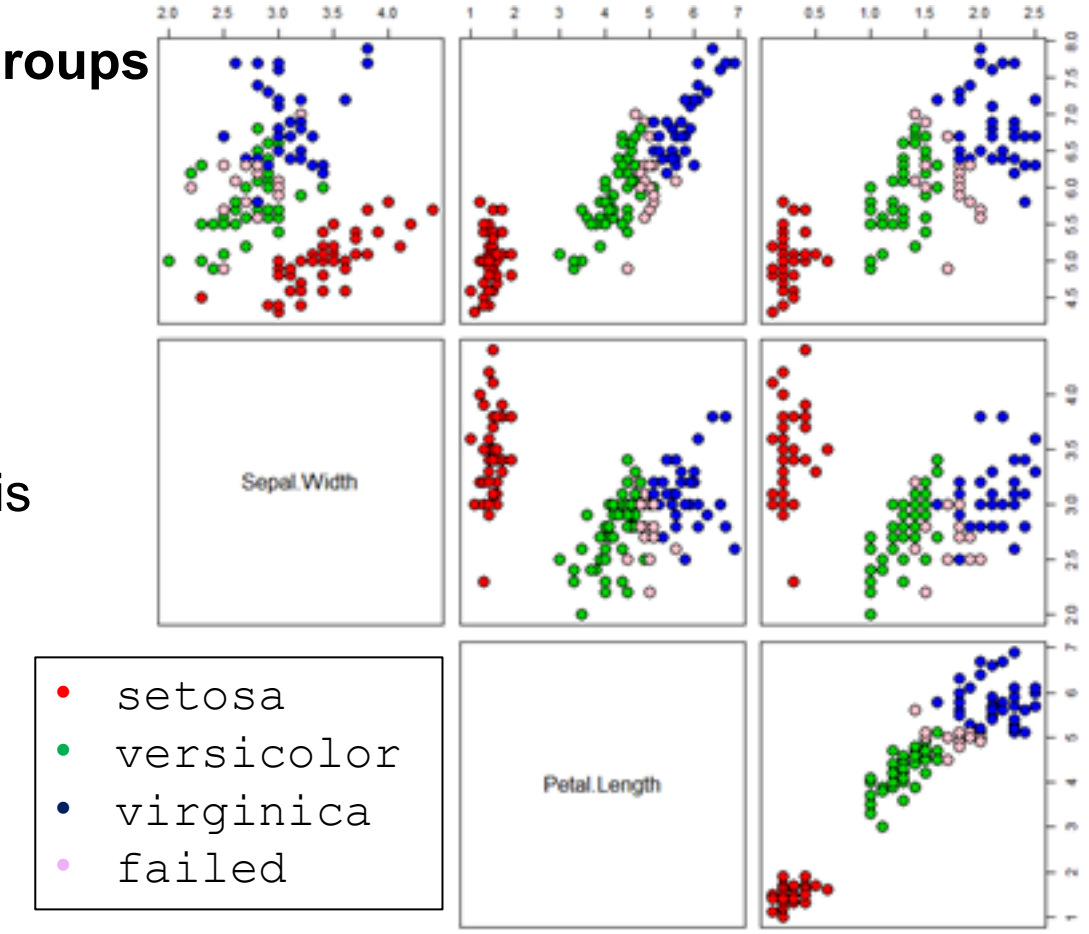


Partitioning methods

K-means

... some instances were assigned to different groups

- The clustering does not fully correspond to the species of the flowers.
- However, that does not necessarily mean that the clusters are not good. (because: it is NOT the task of a cluster analysis to find groups that were previously known)



Partitioning methods

K-Means

Evaluation

- For a good initialization, “good” clusters are found after few steps
- The result is sensitive to the initial cluster centers that were chosen randomly, therefore the clustering is done several times
- Sensitive to outliers since they influence the clusters’ mean values
- We have to be able to calculate the “mean” of a cluster

Alternative

- K-medoids: Similar to K-means but uses actual data points (medoids) as centers, robust to outliers.

Hierarchical methods

Hierarchical methods create a hierarchy of splits/merges of a dataset:
agglomerative methods (bottom-up):

1. start with each data object being one cluster
2. iteratively merge the clusters
3. stop when all clusters are merged or a stopping condition is met

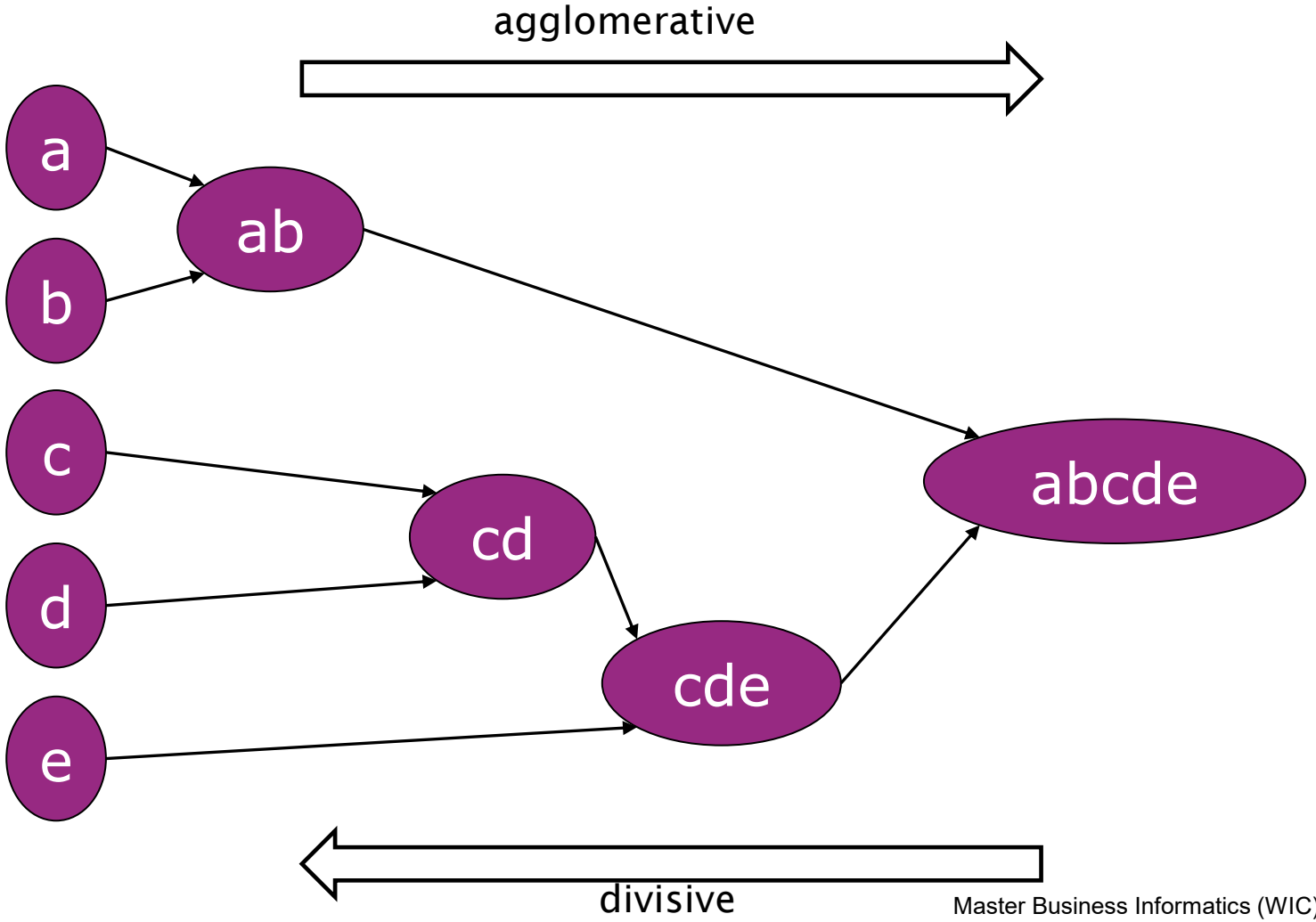
divisive methods (top-down):

1. start with all data objects being one cluster
2. iteratively split each cluster into smaller ones
3. stop when each object forms its own cluster or a stopping condition is met

- Merging or splitting is done based on dissimilarities (= distances, so-called “linkages”) or on densities
- A merging or splitting step can not be undone

Hierarchical methods

Example of hierarchical methods

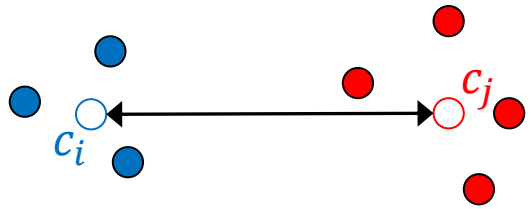


Hierarchical methods

Linkage variants

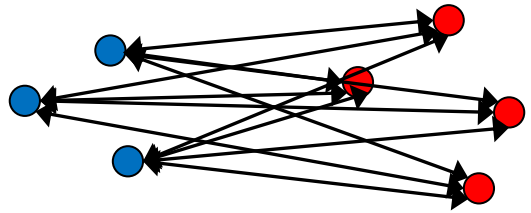
- **Centroid:** Dissimilarity between the centroids, e.g. the mean value vectors, of the two clusters.

$$d(C_i, C_j) = d(c_i, c_j)$$



- **Average linkage:** Average dissimilarity between all pairs of points of the two clusters.

$$d(C_i, C_j) = \frac{1}{N_i N_j} \sum_{o \in C_i} \sum_{o' \in C_j} d(o, o')$$

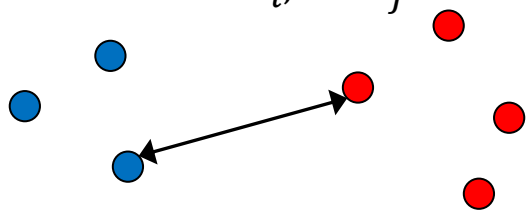


Hierarchical methods

Linkage variants

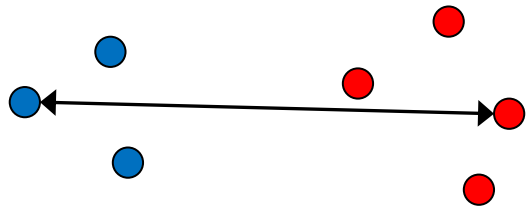
single linkage: Dissimilarity between the two most similar data objects of the two clusters.

$$d(C_i, C_j) = \min_{o \in C_i,\, o' \in C_j} d(o, o')$$



complete linkage: Dissimilarity between the two most dissimilar data objects of the two clusters.

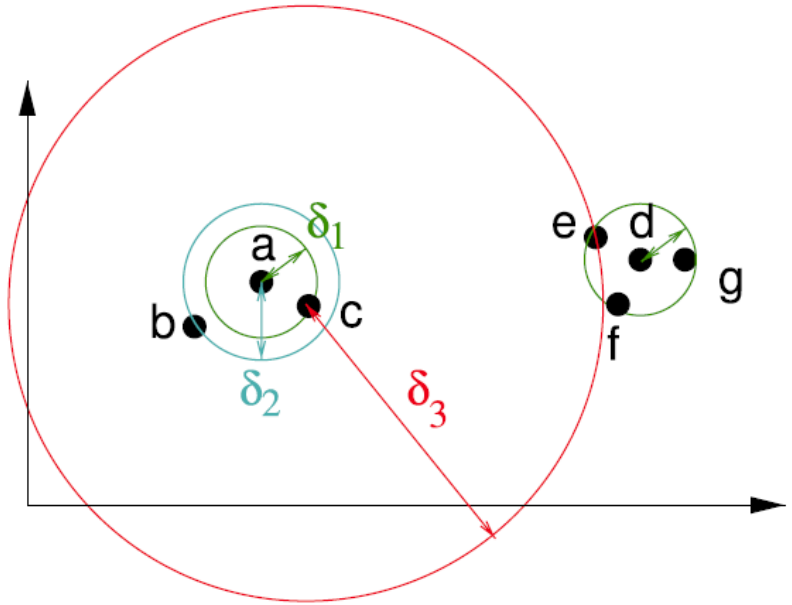
$$d(C_i, C_j) = \max_{o \in C_i,\, o' \in C_j} d(o, o')$$

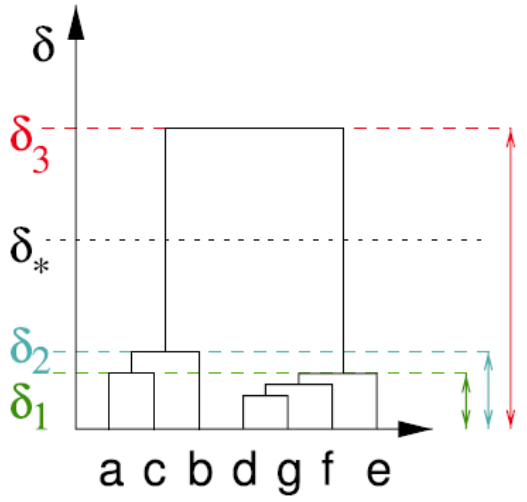


Hierarchical methods

Dendrogram

- Hierarchical clustering can be visualized with a tree-like structure
  - A so-called dendrogram
- The dendrogram shows the splitting/merging operations, together with the corresponding dissimilarities (height = distance)





{ a , c , b , d , g , f , e }

{ {a , c , b}, {d , g , f , e}}

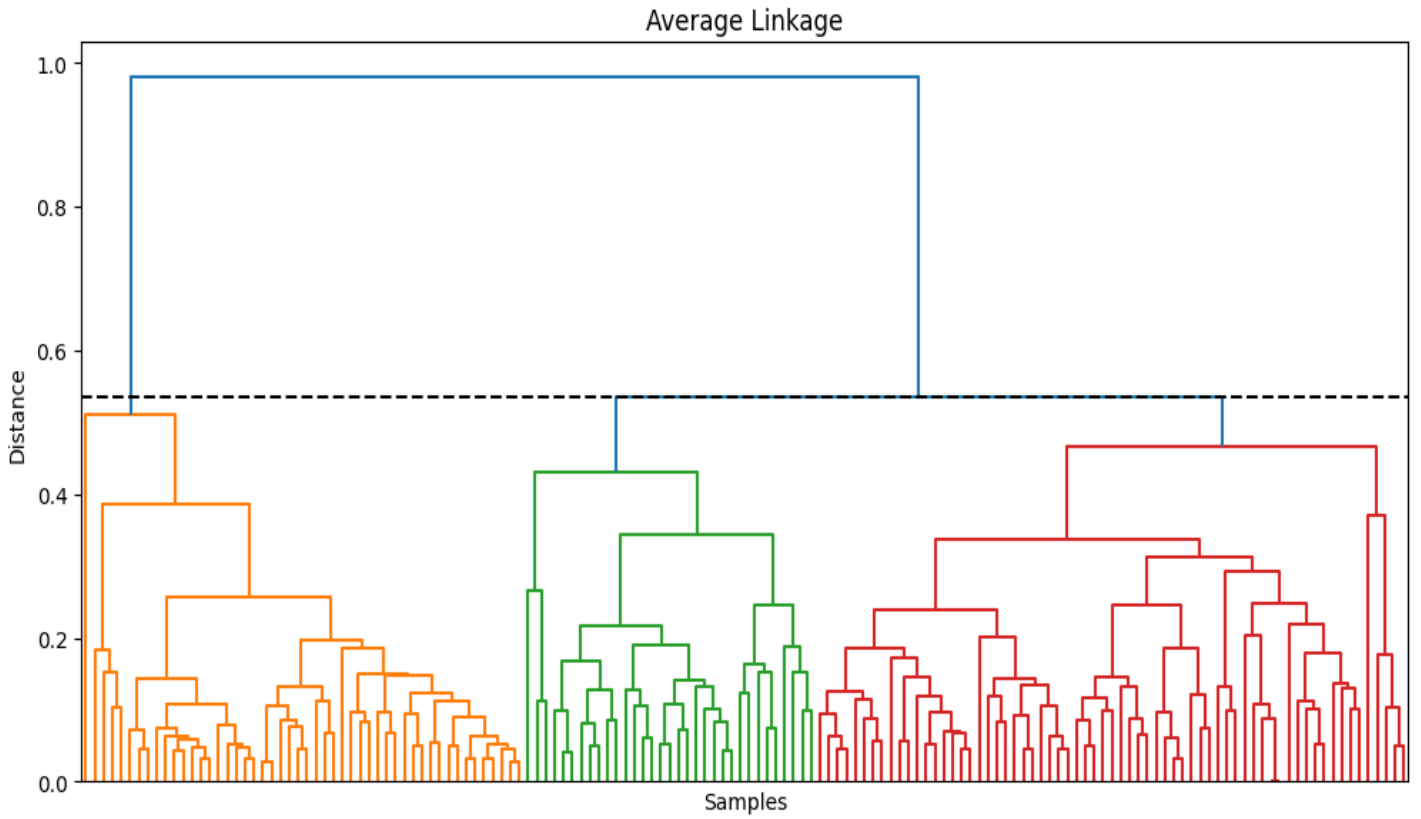
{ {a , c},{b}, {d , g , f , e}}

{ {a},{c},{b},{d},{g},{f},{e}}

Hierarchical methods

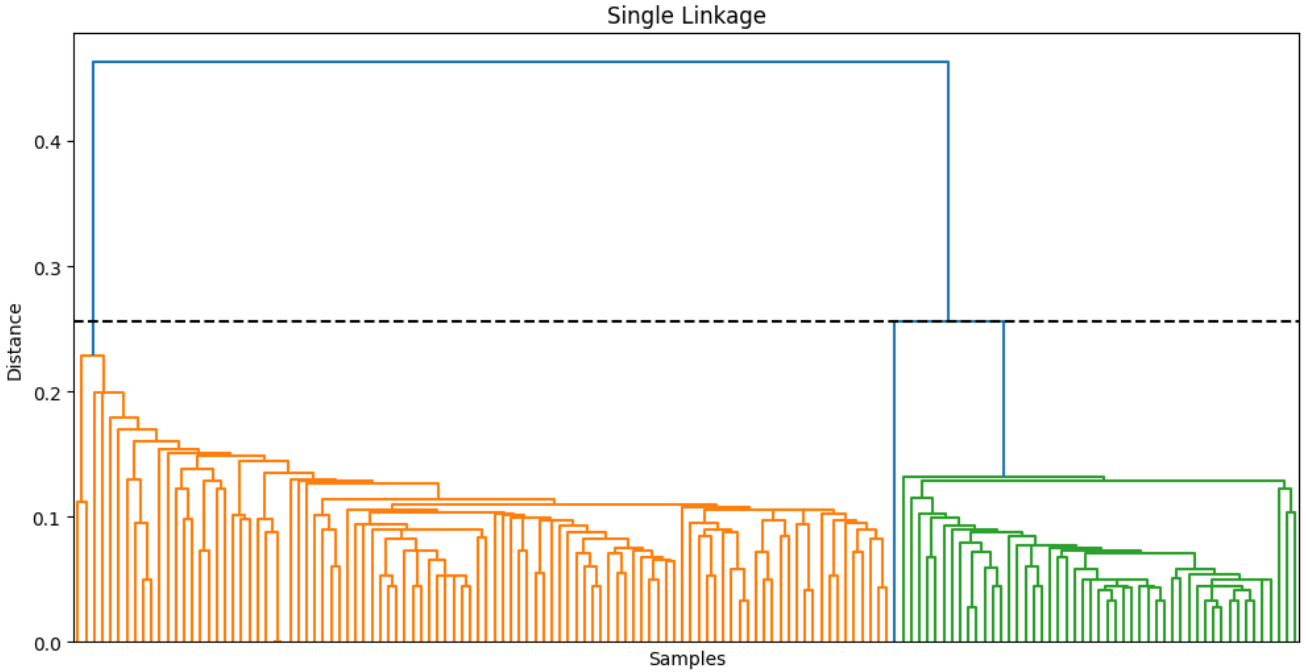
Hierarchical agglomerative clustering

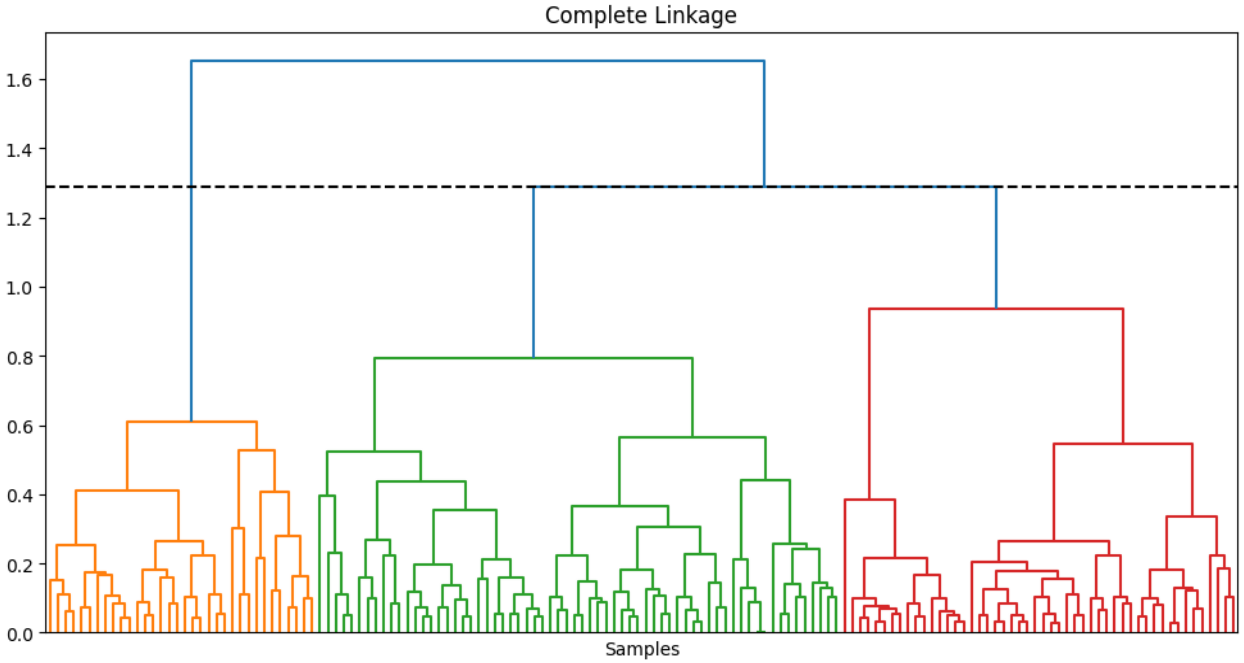
```
# Load the Iris dataset
iris = load_iris()
X = iris.data

# Min-max normalization
scaler = MinMaxScaler()
X_norm = scaler.fit_transform(X)

# Hierarchical agglomerative clustering
model = AgglomerativeClustering(
    n_clusters=3,
    metric="euclidean",
    linkage="average"
)
model.fit(X_norm)
```



Hierarchical methods

Hierarchical agglomerative clustering





Hierarchical methods

Hierarchical agglomerative clustering

Evaluation

- No random initialization is needed
- The dendrogram makes the clustering process easy to interpret
- The result depends on the chosen distance metric and linkage method
- More computationally expensive than K-means for large datasets
- Sensitive to noise and outliers
- Once clusters are merged, the decision cannot be undone

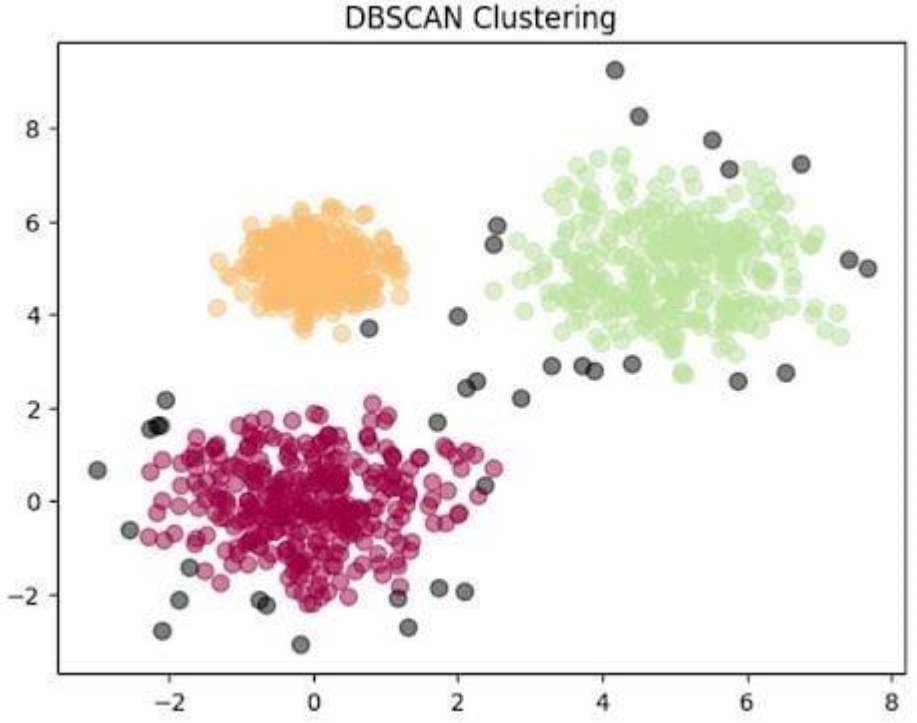
Alternative

- DIANA (Divisive Analysis Clustering): Starts with one large cluster and recursively splits it
- Bisecting K-means: Divides clusters step by step using repeated K-means splits

Density-based methods

DBSCAN

- Groups data points that are closely packed together and marks outliers as noise based on their density in the feature space.
- It identifies clusters as dense regions in the data space separated by areas of lower density, by handling noise points without assigning them to any cluster.



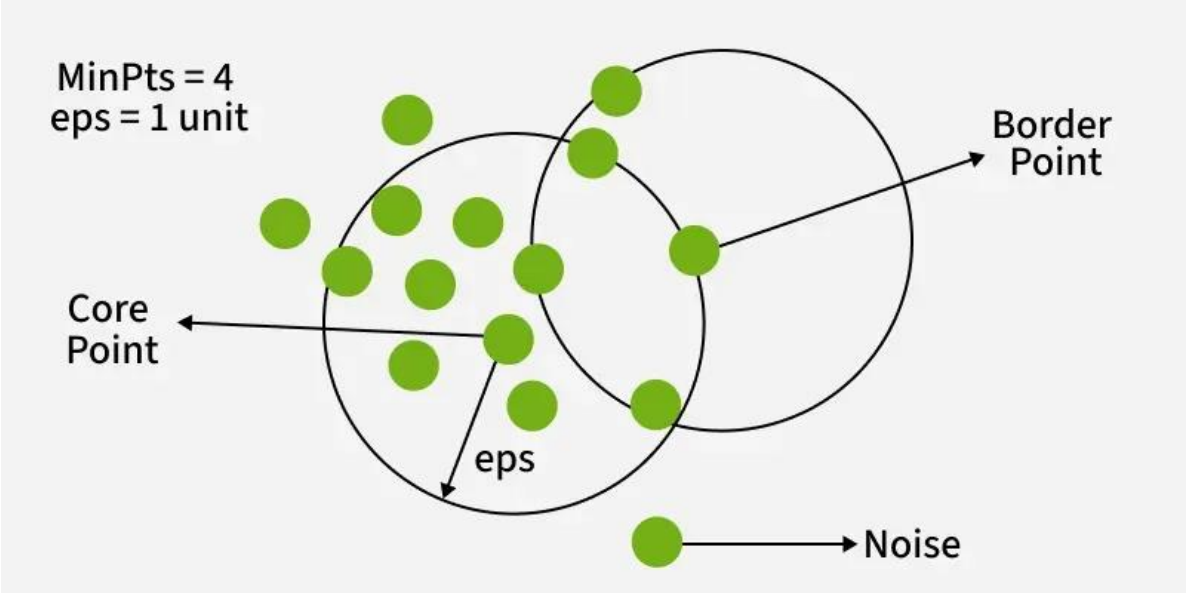
Density-based methods
DBSCAN

Key Parameters:

- eps: Defines the radius of the neighborhood around a data point
- MinPts: Minimum number of points required within the eps radius to form a dense region

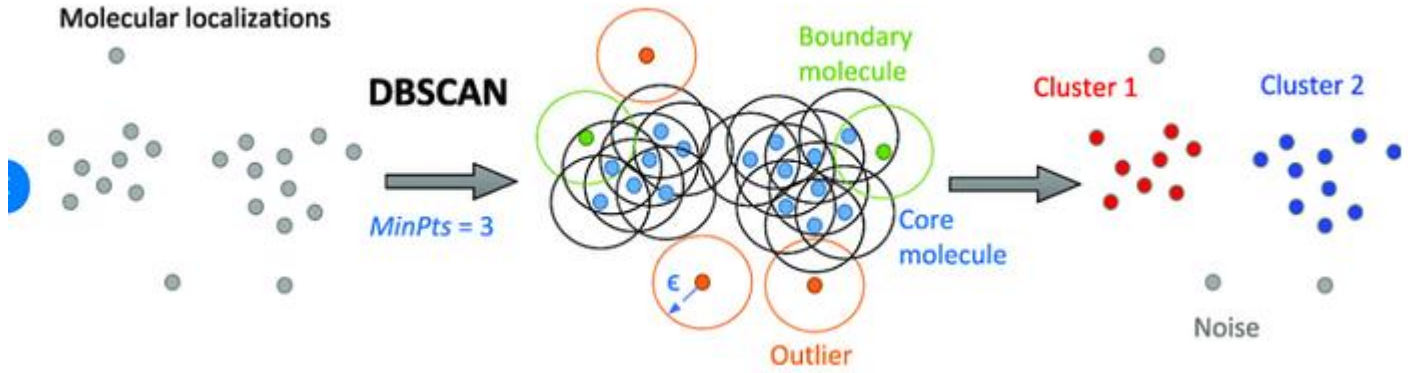
DBSCAN works by categorizing data points into three types:

- Core points which have a sufficient number of neighbors within a specified radius (eplison)
- Border points which are near core points but lack enough neighbors to be core points themselves
- Noise points which do not belong to any cluster



Density-based methods

DBSCAN

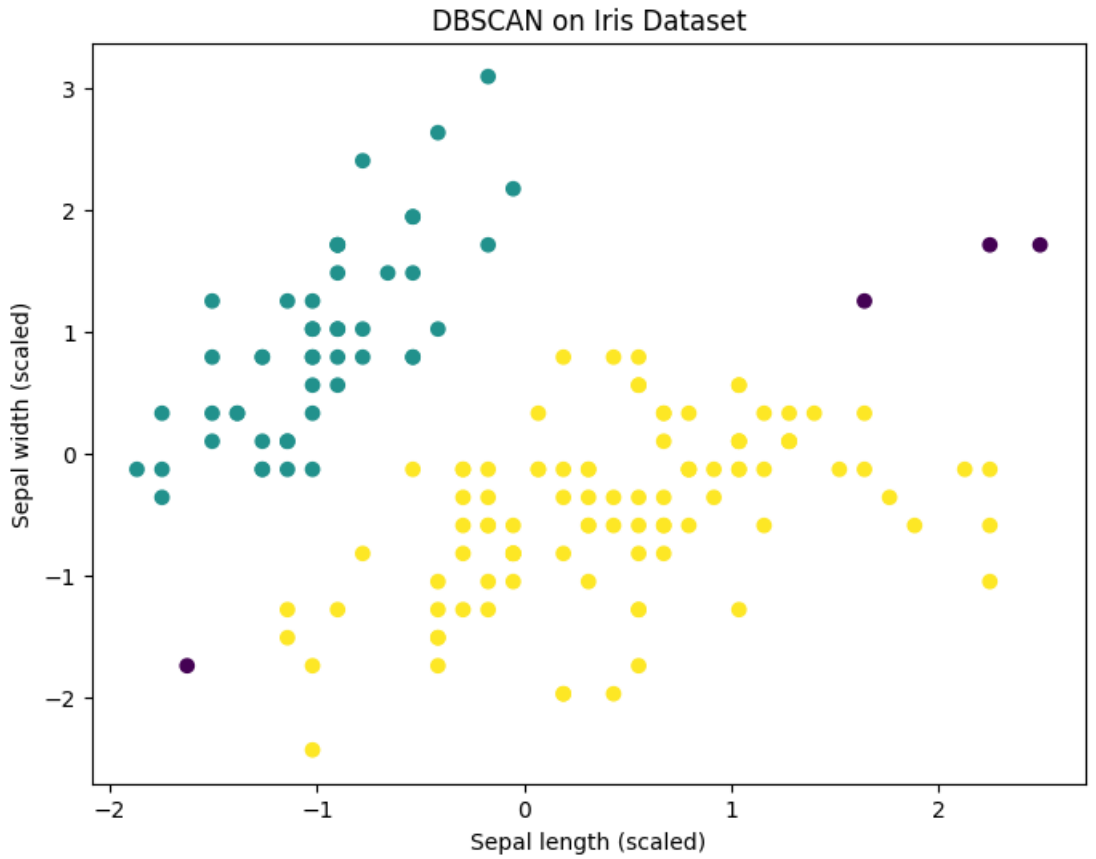


1. **Identify Core Points:** For each point in the dataset count the number of points within its eps neighborhood. If the count meets or exceeds MinPts mark the point as a core point.
2. **Form Clusters:** For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the eps radius of the core point and add them to the cluster.
3. **Density Connectivity:** Two points a and b are density-connected if there exists a chain of points where each point is within the eps radius of the next and at least one point in the chain is a core point. This chaining process ensures that all points in a cluster are connected through a series of dense regions.
4. **Label Noise Points**: After processing all points any point that does not belong to a cluster is labeled as noise.

Density-based methods

DBSCAN

```
# Load the Iris dataset
iris = load_iris()
X = iris.data

# Min-max normalization
scaler = MinMaxScaler()
X_norm = scaler.fit_transform(X)


# Apply DBSCAN
model = DBSCAN(eps=0.5, min_samples=4)
labels = model.fit_predict(X_scaled)
```



Density-based methods

DBSCAN

Evaluation

- No random initialization is needed
- No predefined number of clusters is required
- Can find clusters with irregular shapes
- Noise and outliers are detected automatically
- The result depends strongly on the choice of eps and min_samples
- Struggles when clusters have different densities

Alternative

- OPTICS: Density-based clustering method that handles varying densities better than DBSCAN
- HDBSCAN: Hierarchical extension of DBSCAN with improved robustness

Evaluation of clustering

1. Feasibility of clustering (before clustering)
   - Assessing clustering tendency (Check whether a nonrandom structure exists in the data
   - Determining the number of clusters in a data set

2. Quality of the results (after clustering)
   - Determining the number of clusters in a data set
   - Measuring the clustering quality

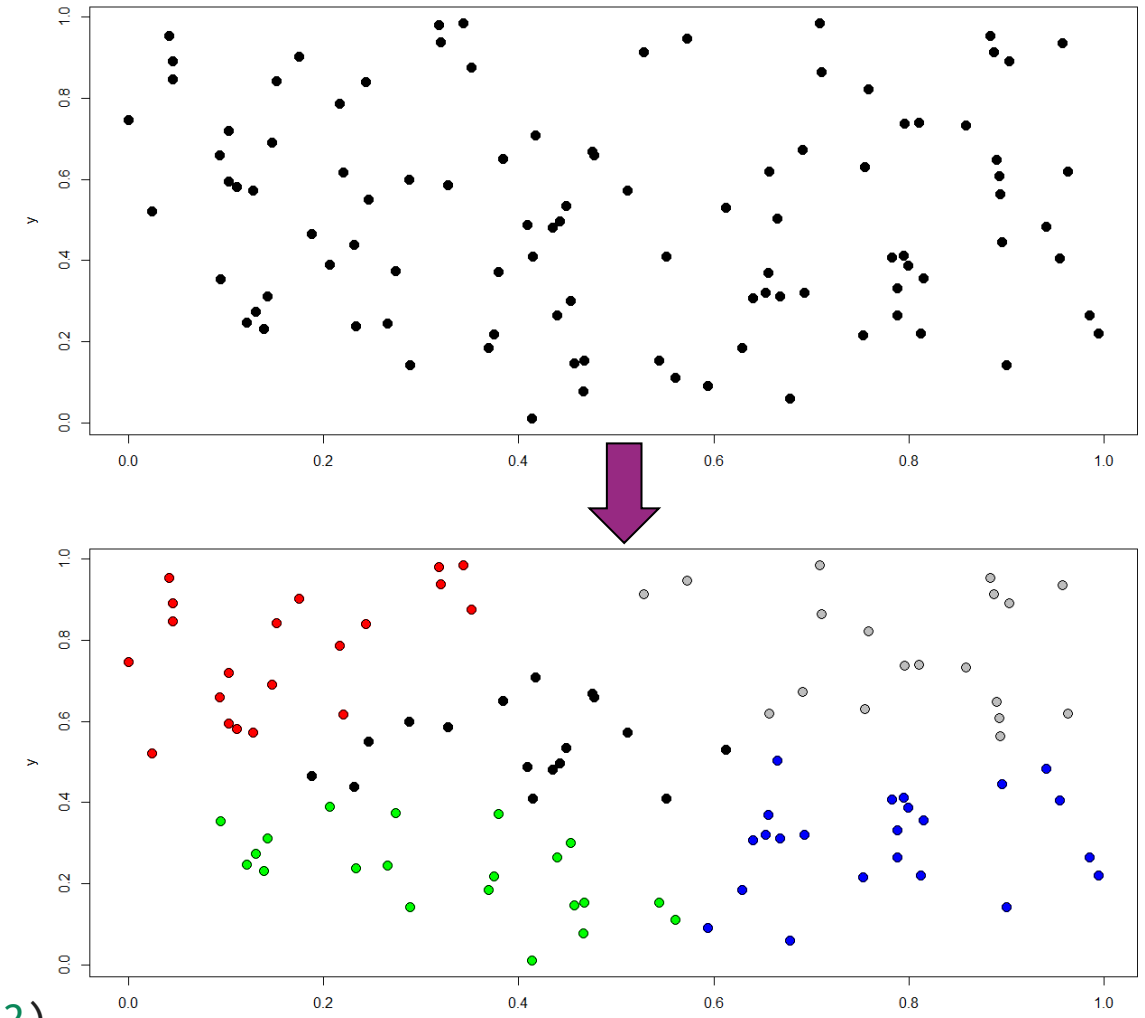
Evaluation of clustering

Feasibility of clustering

Assessing of cluster tendency (before clustering)

- The problem: Algorithms will always find clusters
- But does the data set really have clusters?
- We want to find out if there are clusters in the data.

```
# k-means on random data set
d = pd.DataFrame({
    "x": np.random.uniform(low=0, high=1, size=100),
    "y": np.random.uniform(low=0, high=1, size=100)
})


# k-means with k=5 to the data
cl = KMeans(n_clusters=5, n_init=100, random_state=42)
cl.fit(d)
```



Just because a clustering algorithm produces clusters doesn't mean the data is actually clusterable.

Evaluation of clustering

Determining the number of clusters

For partitioning methods:

- Determining the number of clusters is important, because partitioning methods (e.g. k-Means) require the number of clusters as an input parameter
- "right" number of clusters can be ambiguous
- A **rule of thumb** method (before clustering): Set $k \approx \sqrt{\frac{N}{2}}$ for a data set of N instances.

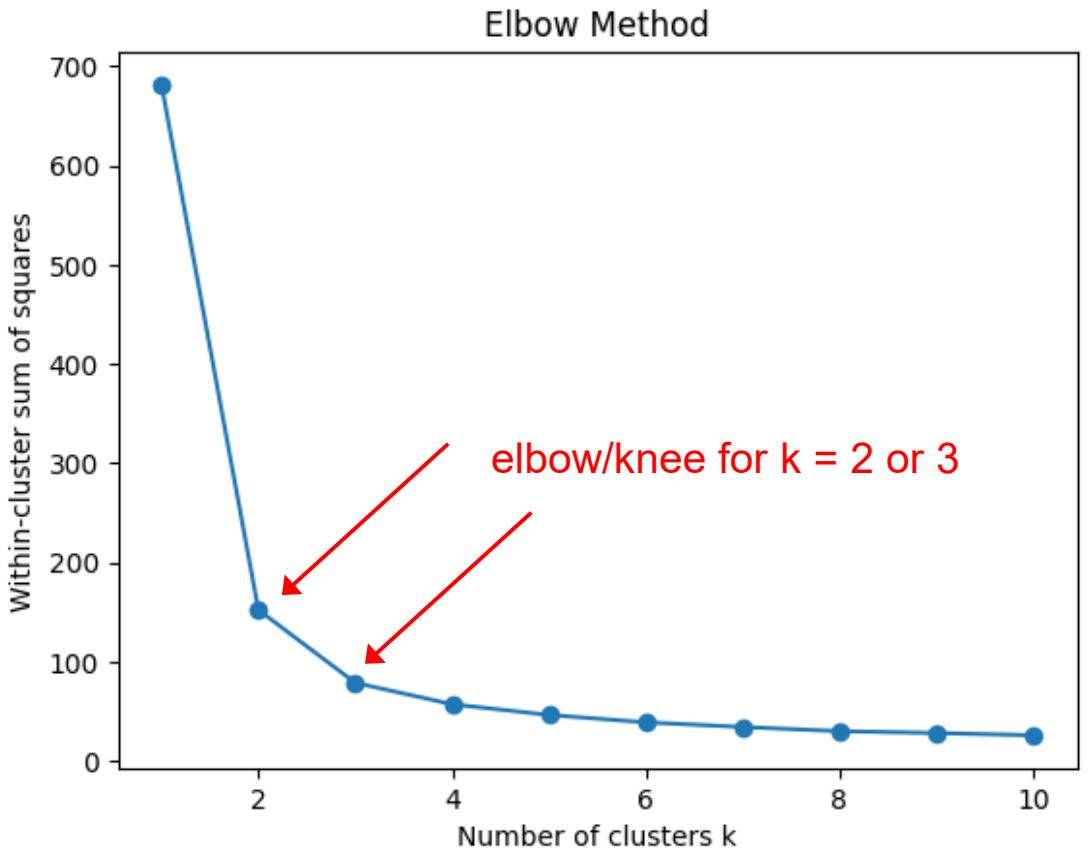
better: elbow/knee method (after clustering)

1. Cluster the data set with k=1,…,N clusters
2. calculate the sum of within-cluster variance $\boldsymbol{E}$ (e.g. sum of squares) of each clustering
3. plot E w.r.t. k and use the turning point (“knee/elbow”)

Evaluation of clustering

Determining the number of clusters

```
X = load_iris().data
errors = []
k_values = range(1, 11)
for k in k_values:
    model = KMeans(n_clusters=k, n_init=10, random_state=42)
    model.fit(X)
    errors.append(model.inertia_)


plt.plot(k_values, errors, marker="o")
plt.xlabel("Number of clusters k")
plt.ylabel("Within-cluster sum of squares")
plt.title("Elbow Method")
plt.show()
```



Evaluation of clustering

Determining the number of clusters

For hierarchical methods:

Options:

- Determine the cutting point from the dendrogram.
- Stop if the distance between the two closest clusters is greater than a pre-defined threshold.
- From the sequence of distances when merging clusters, find a merge step with a significantly higher distance.

Evaluation of clustering

Measuring the cluster quality

Goal: Find out how good a found clustering is.

Following ways are possible:

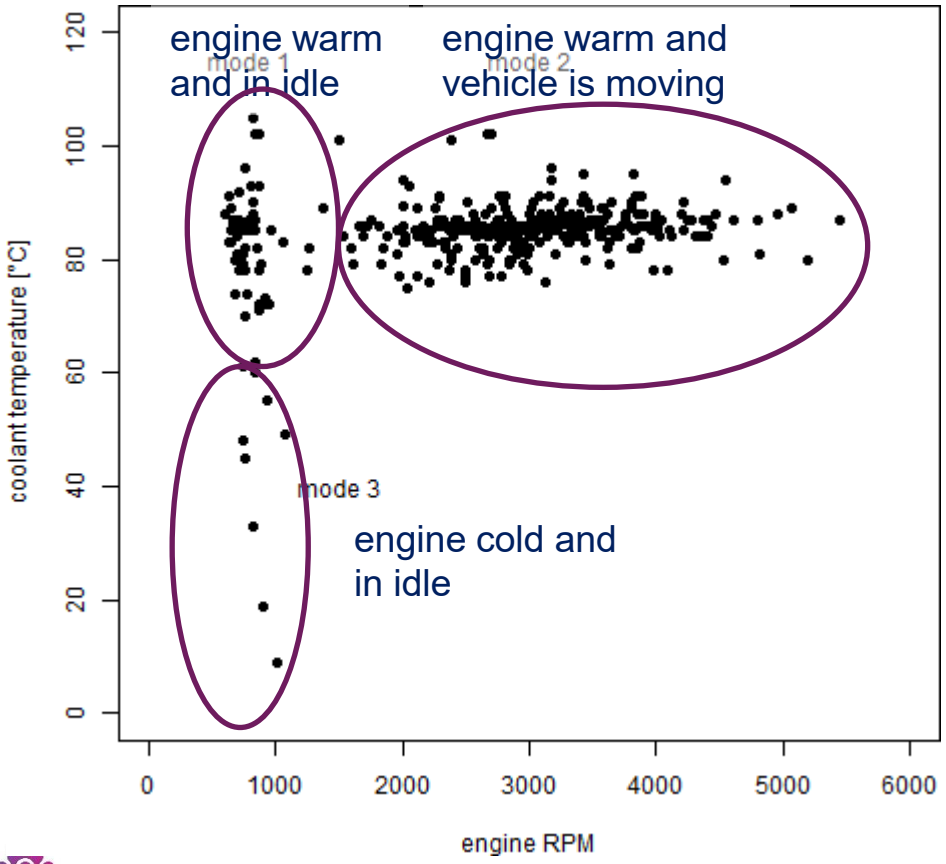
- External criteria ("supervised method"):
  Compare clustering against ground truth (“labels”).

- Internal criteria ("unsupervised method"):
  Evaluate the goodness of a clustering based on the data set, no ground truth required.

- Relative criteria:
  Compare different clusterings (e.g. clusterings using different parameter sets)

Case studie

Cluster vehicle fault codes into the operation modes when they where detected



- With freeze frames engine RPM and engine coolant temperature
- Identified predominant modes:
  - mode 1: engine warm and in idle
  - mode 2: engine warm and vehicle is moving
  - mode 3: engine cold and in idle

Andreas Theissler. “Multi-class Novelty Detection in Diagnostic Trouble Codes from Repair Shops”. Proceedings IEEE International Conference on Industrial Informatics. 2017

Key-Takeaways

- Clustering is an unsupervised learning technique used to discover hidden structure in unlabeled data
- K-means partitions data into a predefined number of clusters, but is sensitive to initialization and outliers
- Hierarchical clustering builds a tree of merges or splits and supports cluster selection through the dendrogram
- DBSCAN groups dense regions, detects noise automatically, and does not require the number of clusters in advance
- Before clustering, cluster tendency should be checked because algorithms can find clusters even in random data
- After clustering, the number and quality of clusters can be evaluated using methods such as the elbow method or cluster quality criteria

Control Questions

1. What is the difference between a core point, a border point, and a noise point in DBSCAN?
2. Why does hierarchical clustering often use a dendrogram instead of requiring a fixed number of clusters from the beginning?
3. What does the elbow method try to detect, and why is the result sometimes ambiguous?
4. Why can the choice of distance metric or linkage method strongly affect hierarchical clustering results?
5. Why can K-means find clusters even in randomly distributed data?
6. In which situations is DBSCAN usually a better choice than K-means?